



GyaanVault

SPPU M.Sc. Data Science — Free Study Material

Lab: Data Mining and Data Warehousing

DS-511-MJP

Solved Practical Manual

Semester I · Lab (MJP)

This solved practical manual covers the Data Mining and Data Warehousing lab assignments from the official SPPU M.Sc. Data Science syllabus. Each practical states the aim followed by a complete, runnable solution and the expected output. Use it as a reference — type the code yourself and experiment with the parameters to learn effectively.

Requirements: Python 3.x with numpy, pandas, matplotlib, scikit-learn, scipy installed (pip install numpy pandas matplotlib scikit-learn scipy).

Practical 1: Scatter plot (Iris dataset)

```
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
d = load_iris()
plt.scatter(d.data[:,0], d.data[:,1], c=d.target, cmap="viridis")
plt.xlabel(d.feature_names[0]); plt.ylabel(d.feature_names[1])
plt.title("Iris – sepal length vs width"); plt.show()
```

Practical 2: Find & remove null values

```
import pandas as pd, numpy as np
df = pd.DataFrame({"A": [1, 2, np.nan, 4], "B": [np.nan, 2, 3, 4]})
print("Nulls per column:\n", df.isnull().sum())
print("After dropna:\n", df.dropna())
```

Practical 3: Encode categorical values to numeric

```
import pandas as pd
df = pd.DataFrame({"city": ["Pune", "Mumbai", "Pune", "Nagpur"]})
print(pd.get_dummies(df, columns=["city"])) # one-hot
df["city_code"] = df["city"].astype("category").cat.codes # label encoding
print(df)
```

Practical 4: Naïve Bayes classifier

```
from sklearn.datasets import load_iris
from sklearn.naive_bayes import GaussianNB
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
X, y = load_iris(return_X_y=True)
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=.3, random_state=0)
print("Accuracy:", accuracy_score(yte, GaussianNB().fit(Xtr, ytr).predict(Xte)))
```

Practical 5: Decision Tree — play tennis

```
import pandas as pd
from sklearn.tree import DecisionTreeClassifier
data = pd.DataFrame({
    "outlook": [0, 0, 1, 2, 2, 2, 1], # 0=sunny 1=overcast 2=rain
    "humidity": [1, 1, 1, 0, 0, 0, 1], # 1=high 0=normal
    "wind": [0, 1, 0, 0, 1, 1, 0],
    "play": [0, 0, 1, 1, 1, 0, 1]})
clf =
DecisionTreeClassifier(criterion="entropy").fit(data[["outlook", "humidity", "wind"]],
```



```
print("Predict [sunny,high,weak]:", clf.predict([[0,1,0]]))
```

Practical 6: Decision boundary with a neural network (two-moons)

```
from sklearn.datasets import make_moons
from sklearn.neural_network import MLPClassifier
X,y = make_moons(n_samples=200, noise=0.2, random_state=0)
clf = MLPClassifier(hidden_layer_sizes=(10,), max_iter=2000).fit(X,y)
print("Train accuracy:", clf.score(X,y))
```

Practical 7: k-Nearest Neighbours prediction model

```
from sklearn.datasets import load_iris
from sklearn.neighbors import KNeighborsClassifier
from sklearn.model_selection import train_test_split
X,y = load_iris(return_X_y=True)
Xtr,Xte,ytr,yte = train_test_split(X,y,test_size=.3,random_state=1)
print("k=3 accuracy:", KNeighborsClassifier(3).fit(Xtr,ytr).score(Xte,yte))
```

Practical 8: k-Means clustering

```
from sklearn.cluster import KMeans
from sklearn.datasets import make_blobs
X,_ = make_blobs(n_samples=200, centers=3, random_state=0)
km = KMeans(n_clusters=3, n_init=10).fit(X)
print("Cluster centres:\n", km.cluster_centers_.round(2))
```

Practical 9: Agglomerative (hierarchical) clustering

```
from sklearn.cluster import AgglomerativeClustering
from sklearn.datasets import make_blobs
X,_ = make_blobs(n_samples=150, centers=3, random_state=2)
labels = AgglomerativeClustering(n_clusters=3).fit_predict(X)
print("First 10 cluster labels:", labels[:10])
```


